

LITERATURE REVIEW

AI AGENTS, MODEL CONTEXT PROTOCOL (MCP) & RETRIEVAL-AUGMENTED GENERATION (RAG)

Phân tích 10 Papers nghiên cứu (2020-2025)

1. Agentic RAG Survey :

1.1. MỤC TIÊU

Vấn đề: RAG truyền thống có quy trình tĩnh, thiếu khả năng thích ứng cho suy luận đa bước và tác vụ phức tạp.

Mục tiêu nghiên cứu:

- Khám phá toàn diện Agentic RAG từ nguyên tắc nền tảng đến ứng dụng thực tế
- Xây dựng taxonomy chi tiết các kiến trúc Agentic RAG
- Vượt qua giới hạn tuyến tính của RAG bằng cách tích hợp AI agents tự chủ

1.2. PHƯƠNG PHÁP

Taxonomy dựa trên 4 Agentic Patterns:

1. Reflection: Tự đánh giá và tinh chỉnh đầu ra
2. Planning: Phân rã tác vụ thành các bước nhỏ
3. Tool Use: Tương tác API, web search, code execution
4. Multi-agent Collaboration: Phối hợp nhiều agents chuyên biệt

6 Kiến trúc chính:

- Single-Agent (Router), Multi-Agent, Hierarchical
- Corrective RAG (CRAG), Adaptive RAG, Graph-Based RAG

1.3. KẾT QUẢ

So sánh với Traditional RAG:

- Có khả năng suy luận đa bước và cộng tác đa tác nhân
- Duy trì trạng thái qua bộ nhớ, tự động điều phối quy trình
- Phù hợp cho các miền kiến thức đa dạng và phức tạp

Ứng dụng thực tế:

- Y tế, tài chính, giáo dục, hỗ trợ khách hàng
- Frameworks: LangChain, LangGraph, LlamaIndex, CrewAI, AutoGen

1.4. ĐIỂM MỚI

- Taxonomy toàn diện nhất về các kiến trúc Agentic RAG
- Unified Framework: Cái nhìn thống nhất về sự hội tụ RAG + Agentic AI
- Industry Mapping: Kết nối lý thuyết với thực tiễn qua use cases cụ thể

1.5. HẠN CHẾ

- Chi phí & độ trễ: Multi-agent và quy trình lặp tốn tài nguyên
- Độ phức tạp: Khó quản lý giao tiếp giữa nhiều agents
- Thiếu benchmark: Chưa có tiêu chuẩn đánh giá chuyên biệt cho Agentic RAG
- Khả năng mở rộng: Tính động có thể quá tải khi khối lượng truy vấn lớn
- Đạo đức AI: Cần giải quyết vấn đề ra quyết định có đạo đức

2. Adaptive-RAG :

2.1. MỤC TIÊU

Vấn đề:

- RAG hiện tại dùng "one-size-fits-all": Single-step thất bại với câu phức tạp, Multi-step lãng phí tài nguyên cho câu đơn giản
- Độ phức tạp truy vấn khác nhau nhưng không được xử lý tương ứng

Mục tiêu:

- Xây dựng hệ thống QA thích ứng động, tự chọn chiến lược phù hợp dựa trên độ phức tạp câu hỏi
- Cân bằng hiệu quả (thời gian/chi phí) và độ chính xác

2.2. PHƯƠNG PHÁP

Classifier-based Routing:

- Mô hình nhỏ phân loại độ phức tạp câu hỏi thành 3 nhãn (A, B, C)

3 Chiến lược:

1. **Label A - No Retrieval:** LLM trả lời trực tiếp (câu hỏi đơn giản)
2. **Label B - Single-step:** Truy xuất 1 lần rồi trả lời (độ phức tạp trung bình)
3. **Label C - Multi-step:** Truy xuất lặp nhiều bước (câu hỏi phức tạp)

Automatic Label Collection (không cần gán nhãn thủ công):

1. Dựa vào kết quả dự đoán: Ưu tiên chiến lược đơn giản nhất trả lời đúng
2. Dựa vào inductive bias: Gán nhãn theo đặc điểm dataset (SQuAD→B, HotpotQA→C)

2.3. KẾT QUẢ

Single-hop QA (SQuAD, NQ, TriviaQA):

- TriviaQA (FLAN-T5-XL): **60.7 F1** vs Adaptive Retrieval 46.9 F1
- Tiệm cận hiệu suất Multi-step mà hiệu quả hơn

Multi-hop QA (MuSiQue, HotpotQA, 2WikiMultiHopQA):

- HotpotQA: **53.82 F1** vs Adaptive Retrieval 32.22 F1, Self-RAG 17.53 F1
- MuSiQue: **31.8 F1** vs Adaptive Retrieval 15.8 F1
- 2WikiMultiHopQA: **49.75 F1** vs Self-RAG 19.59 F1

Efficiency (GPT-3.5):

- F1: **50.91** (ngang Multi-step 50.87)
- Time: **1.46** (nhanh hơn 2.3x so với Multi-step 3.33)

2.4. ĐIỂM MỚI

1. **Query Complexity Classifier:** Phân loại trước khi truy xuất, thay vì dựa vào tần suất thực thể hay suy luận trong quá trình sinh
2. **Automatic Training Data:** Tạo nhãn tự động kết hợp kết quả thực tế + inductive bias, không cần con người
3. **Optimal Balance:** Chuyển đổi mượt mà giữa 3 chiến lược, tránh overkill hoặc underkill

2.5. HẠN CHẾ

- **Phụ thuộc Classifier:** Dự đoán sai → hiệu suất giảm mạnh
- **Gán nhãn tự động:** Inductive bias có thể không chính xác cho mọi mẫu, gây nhiễu

- **Độ mịn phân loại:** Chỉ 3 mức độ (A/B/C), thực tế phức tạp hơn
- **Đạo đức:** Truy vấn độc hại có thể tạo nội dung không phù hợp

3. Self-RAG :

3.1. MỤC TIÊU

Vấn đề:

- Truy xuất bừa bãi: RAG truyền thống truy xuất cố định bất kể cần thiết hay không, gây giảm chất lượng phản hồi
- Thiếu nhất quán và xác thực: Đầu ra không đảm bảo nhất quán với tài liệu vì mô hình không được huấn luyện tuân theo sự kiện

Mục tiêu:

- Nâng cao chất lượng và độ chính xác thực tế của LLM qua truy xuất theo nhu cầu (on-demand) và tự phản ánh
- Huấn luyện LLM tự suy ngẫm quy trình tạo sinh, tự quyết định truy xuất và chọn phân đoạn tốt nhất

3.2. PHƯƠNG PHÁP

4 Reflection Tokens:

1. **Retrieve:** Quyết định khi nào cần truy xuất {yes, no, continue}
2. **IsREL:** Đánh giá tài liệu có liên quan không {relevant, irrelevant}
3. **IsSUP:** Xác minh nội dung có được hỗ trợ bởi tài liệu không {fully/partially/no support}
4. **IsUSE:** Đánh giá mức độ hữu ích {5,4,3,2,1}

Training (2 bước - Knowledge Distillation):

1. **Critic Model:** Huấn luyện trên dữ liệu gán nhãn bởi GPT-4 để dự đoán reflection tokens
2. **Generator Model:** Huấn luyện trên dataset được Critic gán nhãn, học tạo reflection tokens qua next token prediction

Inference - Segment-level Beam Search:

- Xử lý song song nhiều đoạn văn bản truy xuất
- Tạo K ứng viên và đánh giá bằng critique tokens

- Điểm số = tổng trọng số tuyến tính của xác suất critique tokens

3.3. KẾT QUẢ

Open-domain QA:

- PopQA: Self-RAG 13B **55.8** vs Llama2-13B 45.7, ChatGPT 29.3 (no retrieval) / 50.8 (có retrieval)

Fact Verification:

- PubHealth: Self-RAG 13B **74.5**

Long-form Generation:

- ASQA: Self-RAG 7B Citation Precision **66.9**, Recall **67.8**

Reasoning:

- ARC-Challenge: Self-RAG 13B **73.1**

So sánh tổng quát:

- Vượt Llama2-chat và Alpaca trên tất cả tasks
- Vượt ChatGPT trên PopQA, PubHealth, Biography, ASQA

Ablation Studies:

- No Retriever & No Critic → giảm hiệu suất lớn nhất
- Retrieve top 1 (RAG truyền thống) → giảm hiệu suất trên PopQA, ASQA
- Remove IsSUP → giảm hiệu suất ASQA

3.4. ĐIỂM MỚI

1. **Khung Self-RAG**: Mô hình đơn lẻ tự truy xuất, tạo nội dung và tự phê bình qua reflection tokens
2. **Reflection Tokens**: Cơ chế kiểm soát suy luận và đánh giá chất lượng, cho phép tự điều chỉnh hành vi
3. **Controllable Inference**: Điều chỉnh hành vi test-time không cần huấn luyện lại bằng cách thay đổi trọng số w_{rel} , w_{sup} , w_{use}
4. **Hiệu suất vượt trội**: Mô hình nhỏ (7B/13B) vượt ChatGPT và RAG truyền thống trên nhiều tasks

3.5. HẠN CHẾ

- **Chất lượng đầu ra:** Vẫn có thể tạo kết quả không được hỗ trợ hoàn toàn bởi trích dẫn
- **Phụ thuộc:** Phụ thuộc chất lượng Retriever và Critic trong chuẩn bị dữ liệu
- **Chi phí tính toán:** Inference cao hơn do segment-level beam search và xử lý song song nhiều đoạn văn bản (nhưng adaptive retrieval giúp cân bằng)
- **Cải thiện tương lai:** Cần tinh chỉnh retriever, joint training retriever-generator, cải thiện độ chính xác tự phản ánh

4. PaperQA2

4.1. MỤC TIÊU

Vấn đề:

- Kiến thức tham số tĩnh: LLM dựa vào dữ liệu huấn luyện tĩnh, không phản hồi được truy vấn thời gian thực, gây ảo giác
- Knowledge-intensive tasks: Yêu cầu độ chính xác cao, thông tin chi tiết và cập nhật mà mô hình không thể "học thuộc"

Mục tiêu:

- Khắc phục phụ thuộc vào dữ liệu huấn luyện tĩnh bằng cách tích hợp truy xuất dữ liệu thời gian thực
- Kết hợp khả năng tạo sinh của LLM với truy xuất bên ngoài (cơ sở tri thức, API, web) để cung cấp phản hồi phù hợp ngữ cảnh

4.2. PHƯƠNG PHÁP

3 Thành phần chính:

1. **Retrieval:** Truy vấn các nguồn dữ liệu bên ngoài
2. **Augmentation:** Xử lý dữ liệu truy xuất để phù hợp ngữ cảnh truy vấn
3. **Generation:** Kết hợp thông tin truy xuất + kiến thức huấn luyện trước để tạo câu trả lời

Dense Passage Retriever (DPR):

- Biểu diễn truy vấn và tài liệu trong không gian vector nhiều chiều
- Căn chỉnh ngữ nghĩa tốt hơn thay vì chỉ khớp từ khóa

Generator:

- Sử dụng LLM (BART, T5, LLaMA, GPT) để tổng hợp thông tin

Training Approach:

- Cơ chế "Retrieve-Read": Truy xuất đoạn văn bản một lần rồi đưa vào mô hình tạo sinh

4.3. KẾT QUẢ

Open-domain QA:

- PopQA (Llama2-7B): **38.2% - 50.5%** (tùy thiết lập retriever) vs No Retrieval 14.7%
- TriviaQA (Llama2-7B): **42.5%** / Self-RAG reproduction **57.0%**

Natural Questions:

- Single-step RAG (FLAN-T5-XL): **39.0% F1**

Fact Verification:

- PubHealth (Llama2-7B): **30.0%** (thấp hơn Self-RAG 72.4%)

So sánh tổng quát:

- Cải thiện đáng kể so với No Retrieval (PopQA: 38.2% vs 14.7%)

4.4. ĐIỂM MỚI

1. **Kết hợp Retrieval và Generation:** Bước tiến đáng kể kết hợp khả năng tạo sinh LLM với truy xuất thời gian thực
2. **Khả năng thích ứng:** Xử lý truy vấn yêu cầu kiến thức cập nhật/chuyên ngành mà không cần huấn luyện lại toàn bộ mô hình
3. **Nền tảng phát triển:** Đặt nền móng cho Modular RAG, Graph RAG và Agentic RAG

4.5. HẠN CHẾ

- **Truy xuất bừa bãi:** Truy xuất cố định bất kể cần thiết hay không, đưa vào thông tin không liên quan
- **Thiếu nhất quán:** Khó tổng hợp thông tin mạch lạc, gây câu trả lời rời rạc hoặc mâu thuẫn
- **Suy luận đa bước kém:** Single-step thất bại với câu hỏi phức tạp cần suy luận nhiều bước hoặc tổng hợp nhiều tài liệu

- **Chi phí tính toán:** Truy vấn và xếp hạng từ tập lớn gây độ trễ, giảm khả năng ứng dụng thời gian thực
- **Không tự sửa lỗi:** Nếu retriever trả về thông tin sai, mô hình bị "đầu độc" bởi ngữ cảnh sai và tạo ảo giác

5. MCPToolBench++: Large-Scale MCP Tool Use Benchmark

5.1. MỤC TIÊU

Vấn đề:

- Thiếu dataset toàn diện: Không có dataset bao phủ sự đa dạng lớn của MCP tools và JSON schemas
- Định dạng phản hồi đa dạng: Phản hồi từ MCP tool call rất khác nhau (text, image, URL...), khó đánh giá
- Tỷ lệ thành công biến động: Khác benchmark lập trình/toán học, tỷ lệ thành công MCP tool thực tế không đảm bảo (lỗi API, timeout)
- Giới hạn Context Window: Mô tả tool và tham số quá dài, giới hạn số công cụ đưa vào context LLM

Mục tiêu:

- Mở rộng quy mô đánh giá khả năng function calling của LLM trong bối cảnh Model Context Protocol (MCP)
- Tích hợp các nguồn dữ liệu thực tế (search, finance, map...)
- Xây dựng benchmark từ >4,000 MCP servers, bao phủ >40 danh mục, tạo 1.5K cặp câu hỏi-trả lời

5.2. PHƯƠNG PHÁP

Dataset Construction:

- Thu thập schema từ marketplace mở (smithery.ai, deepnlp.org, pulsemcp.com)
- Tool Sampler chọn công cụ + Query Generator (LLM) tạo prompt template
- Parameter Value Generation + Slot Filling

Single-step vs Multi-step:

- Bao gồm câu hỏi đơn bước và đa bước phức tạp
- Tool call chains có thể lên tới 10 công cụ

Evaluation Metrics:

1. **AST (Abstract Syntax Tree)**: Độ chính xác tĩnh của chọn hàm và điền tham số
2. **AST DAG Accuracy**: Đánh giá cấu trúc đồ thị của chuỗi gọi đa bước
3. **Pass@K**: Thực thi thành công và kết quả khớp mong đợi
4. **Tool Call Success Rate**: Tỷ lệ chạy tool không bị lỗi runtime/API

Post-processing:

- Semantic Check: Loại bỏ truy vấn viết lại sai (ví dụ: tọa độ không chuyển đổi được)
- Reasonableness Check: Loại bỏ truy vấn phi thực tế ("Đi tàu NY-Tokyo")

LLMs tested: GPT-4o, Claude-3.7-Sonnet, Qwen2.5-max, Qwen3-coder, Kimi-K2-Instruct

5.3. KẾT QUẢ

AST Accuracy (Tool Selection & Parameter Inference):

- Browser: Qwen3-coder **0.887**
- File System: Qwen2.5-max **0.942**
- Search: Kimi-K2-Instruct **0.732**
- Map: Qwen3-coder **0.783**
- Pay: Kimi-K2-Instruct **0.807**
- Finance: Qwen2.5-max **0.751**

Pass@1 Performance (Execution Success):

- Browser: Qwen3-coder **0.292**
- File System: Qwen2.5-max **0.887**
- Search: Claude-3.7-Sonnet **0.620** (vượt Kimi 0.368 dù AST thấp hơn)
- Map: GPT-4o **0.362**
- Pay: Kimi-K2-Instruct **0.676**

Insights:

- Không có model thắng áp đảo: Qwen mạnh kỹ thuật (Browser, File), Claude-3.7 mạnh Search thực tế, GPT-4o ổn định Map/Finance
- Chênh lệch AST vs Pass@1: Kimi chọn đúng tool nhưng chạy thất bại nhiều; Claude chọn tool ổn định hơn

5.4. ĐIỂM MỚI

1. **Largest MCP Benchmark**: Benchmark quy mô lớn đầu tiên cho MCP với >4,000 servers, tạo 1.5K cặp câu hỏi-đáp chất lượng

2. **Comprehensive Category Coverage:** Bao phủ >40 danh mục (Search, Map, Finance, Browser, File...), vượt xa benchmark trước chỉ tập trung code/math
3. **Real-world Tool Deployments:** Đánh giá trên công cụ thực tế executable, yêu cầu environment + API key cụ thể, thay vì mô phỏng văn bản tĩnh

5.5. HẠN CHẾ

- **MCP Server Stability:** Lỗi API (432, 404, 500), Connection Timeout xảy ra thường xuyên, ảnh hưởng Pass@K
- **Context Window:** Tool schema tiêu tốn nhiều token ($O(MNtT_{tool})$), cần Tool Dispatcher lọc công cụ không liên quan
- **Success Rate Variations:** Chênh lệch lớn giữa AST (chọn đúng) và Pass@1 (chạy thành công)
- **Reasoning Failures:** Lỗi suy luận tham số (ví dụ: tọa độ ngoài phạm vi cho phép trong Map tool)

6. MCP Security and Maintainability Study

6.1. MỤC TIÊU

Vấn đề:

- **Rủi ro bảo mật mới của MCP:**
Model Context Protocol (MCP) cho phép mô hình nền tảng gọi công cụ để tương tác với thế giới thực, nhưng luồng điều khiển không xác định do AI điều phối làm phát sinh các rủi ro bảo mật mới.
- **Nguy cơ tool poisoning:**
Các nghiên cứu sơ bộ cho thấy mô hình có thể bị ép sử dụng các công cụ từ MCP servers để bị tổn thương để tấn công hệ thống người dùng.
- **Thiếu đánh giá quy mô lớn:**
Trước nghiên cứu này, chưa có phân tích thực nghiệm nào đánh giá toàn diện hệ sinh thái MCP servers ở quy mô lớn.

Mục tiêu:

- Đánh giá mức độ an toàn (security) và khả năng bảo trì (maintainability) của MCP servers.
- Phân tích hệ sinh thái MCP trên quy mô lớn với 1.899 MCP servers mã nguồn mở.
- Trả lời ba câu hỏi nghiên cứu chính:
 - Mức độ “khỏe mạnh” và bền vững của MCP servers.
 - Mức độ tồn tại các lỗ hổng bảo mật.
 - Mức độ tồn tại các vấn đề về khả năng bảo trì.

6.2. PHƯƠNG PHÁP

Phương pháp phân tích:

- Áp dụng pipeline phân tích lai (hybrid analysis) kết hợp giữa:
 - Công cụ phân tích tĩnh đa năng SonarQube.
 - Công cụ quét chuyên biệt cho MCP là mcp-scan.

Đánh giá sức khỏe dự án:

- Sử dụng 14 chỉ số (metrics) từ CHAOSS và các nghiên cứu trước, bao gồm:
 - Tần suất commit.
 - Tỷ lệ áp dụng CI (Continuous Integration).
 - Số lượng và danh tiếng người đóng góp.
 - Thời gian xử lý issue.

Phát hiện lỗ hổng:

- SonarQube được sử dụng để quét 583 repositories đã được lọc (loại bỏ các dự án nhỏ dưới 10 sao).
- mcp-scan được áp dụng trên 83 repositories được chọn ngẫu nhiên để phát hiện lỗi tool poisoning.

Phân nhóm vấn đề:

- Sử dụng phương pháp LLM Jury (GPT-4o, Claude 3.7 Sonnet làm worker và Gemini 2.5 Pro làm judge) để gom nhóm các lỗ hổng, code smell và bug.
- Xác định tổng cộng 8 loại lỗ hổng thông qua phân tích và clustering.

6.3. KẾT QUẢ

Sức khỏe hệ sinh thái MCP:

- MCP servers có mức độ phát triển tích cực với trung bình 5.5 commits/tuần, cao hơn phần mềm truyền thống.
- Tỷ lệ áp dụng CI đạt 42.2%, nhỉnh hơn mức trung bình của các dự án truyền thống.
- Các MCP servers được khai thác từ cộng đồng có quy mô lớn hơn và hoạt động phát triển sôi động hơn các server chính thức.

Bảo mật:

- 7.2% MCP servers chứa các lỗ hổng bảo mật chung.
- 5.5% MCP servers chứa lỗ hổng đặc thù MCP là tool poisoning.
- Credential Exposure là lỗ hổng phổ biến nhất, chiếm 3.6%.
- Chỉ 3/8 loại lỗ hổng trùng với các lỗ hổng phần mềm truyền thống, cho thấy MCP có hồ sơ rủi ro riêng biệt.

Khả năng bảo trì:

- 66% MCP servers chứa ít nhất một code smell ở mức Critical hoặc Blocker.
- High Cognitive Complexity là vấn đề phổ biến nhất, ảnh hưởng đến 59.7% servers.
- 14.4% servers chứa bug nghiêm trọng, phổ biến nhất là lỗi thao tác mảng.

6.4. ĐIỂM MỚI

- Nghiên cứu MCP quy mô lớn đầu tiên: Đây là nghiên cứu thực nghiệm đầu tiên đánh giá toàn diện hệ sinh thái MCP servers trên quy mô lớn.
- Chỉ ra rủi ro đặc thù MCP: MCP servers không gặp các vấn đề phổ biến như XSS hay gói độc hại, mà tập trung vào lộ credential, thiếu kiểm soát truy cập và tool poisoning.
- Phương pháp phân tích lai: Kết hợp hiệu quả công cụ truyền thống và công cụ chuyên biệt, thiết lập baseline cho các nghiên cứu MCP trong tương lai.

6.5. HẠN CHẾ

- Giới hạn của phân tích tĩnh: SonarQube và mcp-scan có thể bỏ sót các lỗ hổng phụ thuộc ngữ cảnh hoặc hành vi runtime.
- Thiếu phân tích động: Nghiên cứu chưa thực hiện phân tích động hay kiểm thử thâm nhập, nên chưa phát hiện được các lỗi logic phức tạp.
- Giới hạn tính khái quát: Dữ liệu dựa trên MCP servers mã nguồn mở, có thể không đại diện cho các server thương mại hoặc riêng tư.

7. MCP Safety Audit

7.1. MỤC TIÊU

Vấn đề:

- **Phát hiện các hình thức khai thác bảo mật:**
Nghiên cứu chứng minh rằng các Mô hình Ngôn ngữ Lớn (LLM) hàng đầu có thể bị ép buộc sử dụng các công cụ từ các máy chủ MCP tiêu chuẩn để thực

hiện ba loại tấn công chính:

- **Malicious Code Execution (MCE):** Thực thi mã độc (ví dụ: chèn mã vào file hệ thống).
- **Remote Access Control (RAC):** Giành quyền truy cập từ xa vào hệ thống nạn nhân.
- **Credential Theft (CT):** Đánh cắp thông tin nhạy cảm như API key từ file hệ thống hoặc biến môi trường.
- **Mức độ nghiêm trọng của lỗ hổng:**
Các lỗ hổng này cho phép kẻ tấn công chiếm quyền kiểm soát hệ thống của nhà phát triển AI hoặc đánh cắp dữ liệu nhạy cảm.

Mục tiêu:

- Chứng minh khả năng khai thác thực tế các MCP servers tiêu chuẩn thông qua LLM.
- Xây dựng **mcpSafetyScanner**, công cụ agentic đầu tiên nhằm:
 - Tự động đánh giá bảo mật của bất kỳ MCP server nào.
 - Phát hiện các mẫu tấn công đối kháng.
 - Cung cấp báo cáo bảo mật và biện pháp khắc phục trước khi triển khai.

7.2. PHƯƠNG PHÁP

Kiến trúc mcpSafetyScanner:

Công cụ sử dụng kiến trúc **đa tác nhân (multi-agent)** gồm ba giai đoạn:

1. Hacker Agent:

Tự động tải xuống các thành phần của MCP server (công cụ, tài nguyên, prompt) và thăm dò hệ thống để phát hiện lỗ hổng.

2. Auditor Agent:

Phân tích các lỗ hổng phát hiện được, tra cứu các cơ sở tri thức (Web, ArXiv,

Hacker News) để tìm lỗ hổng tương tự và đề xuất biện pháp khắc phục.

3. Supervisor Agent:

Tổng hợp kết quả và sinh báo cáo bảo mật chi tiết.

Kịch bản tấn công được đánh giá:

- **Direct Prompt Attacks (DPA):**

Trực tiếp yêu cầu LLM (Claude 3.7, Llama-3.3-70B) thực hiện hành vi độc hại thông qua MCP tools.

- **Retrieval-Agent Deception (RADE):**

Một hình thức tấn công mới, trong đó dữ liệu công khai bị nhiễm độc và đưa vào vector database, kích hoạt các lệnh MCP độc hại khi người dùng truy vấn.

Khai thác minh chứng (Proof-of-Concept):

- **MCE:** Chèn lệnh netcat vào file `.bashrc` thông qua MCP Filesystem tool.
- **RAC:** Thêm SSH key của kẻ tấn công vào `~/.ssh/authorized_keys`.
- **CT:** Trích xuất API key từ biến môi trường hoặc file hệ thống và gửi ra ngoài qua Slack.

7.3. KẾT QUẢ

Các khai thác chính:

- Cả **Claude 3.7** và **Llama-3.3-70B** đều có thể bị khai thác thành công để thực hiện MCE, RAC và CT thông qua các MCP servers tiêu chuẩn.

Hành vi mô hình và tỷ lệ thành công:

- **Claude 3.7:**
Đôi khi kích hoạt guardrails và từ chối các lệnh quá rõ ràng, nhưng có thể bị vượt qua bằng prompt đơn giản hoặc lệnh văn bản thuần.
- **Llama-3.3-70B:**
Thường xuyên hoàn thành yêu cầu tấn công ngay cả khi nhận diện được rủi ro;

chỉ từ chối khi prompt chứa các từ khóa nguy hại rõ ràng.

Đánh giá tác động:

- **RADE** đặc biệt nguy hiểm vì cho phép chiếm quyền truy cập hoặc đánh cắp dữ liệu chỉ thông qua việc người dùng truy vấn vector database bị nhiễm độc.

Hiệu quả của mcpSafetyScanner:

- Công cụ phát hiện chính xác các lỗ hổng, cung cấp ví dụ khai thác cụ thể và đề xuất biện pháp khắc phục tương ứng.

7.4. ĐIỂM MỚI

- **Giới thiệu công cụ audit bảo mật cho MCP:**
McpSafetyScanner là công cụ agentic đầu tiên tự động đánh giá bảo mật cho MCP servers.
- **Phát hiện vector tấn công RADE:**
Đề xuất và chứng minh một hình thức tấn công mới thông qua dữ liệu trong vector database.
- **Minh chứng khai thác thực tế:**
Chứng minh rằng các LLM hàng đầu kết hợp với MCP servers phổ biến tạo ra rủi ro bảo mật nghiêm trọng.
- **Đề xuất biện pháp khắc phục thực tiễn:**
Cung cấp khuyến nghị và lỗi cụ thể cho từng loại tấn công.

7.5. HẠN CHẾ

- **Giới hạn phạm vi:**
Nghiên cứu tập trung vào một số MCP servers phổ biến; các server tùy chỉnh khác chưa được đánh giá.
- **Tính ngẫu nhiên của agent:**
Do bản chất stochastic, các lần quét khác nhau có thể phát hiện hoặc bỏ sót lỗ

hồng.

- **Phụ thuộc vào guardrails:**

Việc dựa hoàn toàn vào guardrails của LLM là không đủ an toàn do dễ bị vượt qua bằng prompt engineering.

Khuyến nghị khắc phục:

- **MCE:** Kiểm soát truy cập tệp nghiêm ngặt, giám sát thay đổi file hệ thống.
- **RAC:** Áp dụng phân quyền chặt chẽ cho `authorized_keys`.
- **CT:** Không lưu thông tin nhạy cảm trong biến môi trường, tuân thủ nguyên tắc đặc quyền tối thiểu.
- **Slack:** Hạn chế quyền API và thường xuyên rà soát quyền truy cập.

8. MCP-Bench: Benchmarking Tool-Using LLM Agents

8.1. MỤC TIÊU

Định hướng nghiên cứu:

- Nghiên cứu tập trung đánh giá khả năng của các **LLM Agent** trong việc giải quyết các **tác vụ thực tế phức tạp**, đòi hỏi nhiều bước xử lý liên tiếp.
- Các tác vụ yêu cầu agent phải:
 - Sử dụng và phối hợp nhiều công cụ khác nhau.
 - Kiểm soát tham số chính xác.
 - Thực hiện lập kế hoạch và suy luận trong quá trình thực thi.

Khác biệt so với các benchmark trước:

- Các benchmark cũ như ToolBench hay BFCL thường sử dụng các API cô lập, dẫn đến quy trình ngắn, thiếu liên kết tự nhiên giữa các công cụ.

- MCPToolBench++ chủ yếu tập trung vào quy mô lớn và đánh giá tĩnh.
- MCP-Bench tập trung vào **độ sâu và mức độ phức tạp của workflow**, yêu cầu agent:
 - Tự tìm công cụ từ các chỉ dẫn mơ hồ.
 - Không được chỉ định sẵn tên công cụ như các benchmark truyền thống.

Quy mô:

- Kết nối LLM với **28 MCP servers thực tế**, bao gồm **250 công cụ được cấu trúc hóa**, hoạt động trong môi trường sống (live).

8.2. PHƯƠNG PHÁP

Thiết kế tác vụ:

- Áp dụng quy trình tổng hợp tác vụ dựa trên LLM gồm ba giai đoạn:
 1. **Phát hiện chuỗi phụ thuộc:** Phân tích đầu vào và đầu ra của công cụ để xác định các chuỗi logic giữa các công cụ.
 2. **Lọc chất lượng tự động:** Đảm bảo tác vụ có khả năng giải được cao và có giá trị sử dụng thực tế.
 3. **Làm mờ mô tả tác vụ:** Chuyển hướng dẫn kỹ thuật thành mô tả ngôn ngữ tự nhiên, mơ hồ, buộc agent phải tự suy luận và lập kế hoạch.

Đánh giá workflow đa bước:

- Các tác vụ yêu cầu quy trình đa vòng, có thể lên tới **20 vòng thực thi**.
- Agent phải duy trì trạng thái và điều phối giữa nhiều MCP server khác nhau.

Các mô hình được đánh giá:

- Tổng cộng **20 LLM**, bao gồm các mô hình hàng đầu như GPT-5, o3, GPT-4o, Claude 3.5 Sonnet, Gemini 2.5 Pro, Llama 3, Qwen và các mô hình liên quan.

Khung đánh giá hai tầng:

- **Tầng quy tắc:** Kiểm tra tính hợp lệ của tên công cụ, tuân thủ schema và tỷ lệ thực thi thành công.
- **Tầng đánh giá bằng LLM:** Chấm điểm dựa trên mức độ hoàn thành tác vụ, cách sử dụng công cụ và hiệu quả lập kế hoạch.

Lĩnh vực ứng dụng:

- Bao phủ **11 lĩnh vực thực tế**, bao gồm tài chính, khoa học, y tế, địa lý, truyền thông và giải trí.

8.3. KẾT QUẢ

Hiệu năng tổng thể của các mô hình:

- Nhóm dẫn đầu gồm GPT-5, o3 và gpt-oss-120b với điểm số cao nhất.
- Nhóm mạnh gồm Gemini 2.5 Pro, Claude 3.5 Sonnet và Qwen3-235B.
- Các mô hình nhỏ như Llama-3.1-8b-instruct đạt hiệu năng thấp nhất.

Khả năng xử lý tác vụ đa bước:

- GPT-5 và o3 thể hiện khả năng lập kế hoạch và nhận thức phụ thuộc vượt trội.
- Các mô hình nhỏ hiếm khi đạt được hiệu quả trong các tác vụ đa bước phức tạp.

Hiệu quả và mức tiêu thụ tài nguyên:

- Các mô hình mạnh hoàn thành tác vụ với số vòng và số lần gọi công cụ ít hơn.

- Các mô hình yếu thường rơi vào vòng lặp, tiêu tốn nhiều tài nguyên nhưng hiệu quả thấp.

So sánh với benchmark đơn giản:

- Khả năng tuân thủ schema đã đạt mức bão hòa.
- Sự khác biệt chính giữa các mô hình nằm ở khả năng suy luận bậc cao và thích ứng trong môi trường đa server.

8.4. ĐIỂM MỚI

- Benchmark được phát triển theo **tiêu chuẩn cấp doanh nghiệp**, với sự hợp tác giữa Accenture và UC Berkeley.
- Sử dụng **hệ sinh thái công cụ thực tế**, kết nối trực tiếp với các MCP servers đang hoạt động.
- Đóng góp quan trọng là việc đánh giá khả năng của agent khi xử lý các yêu cầu mơ hồ, không chỉ định sẵn công cụ, phản ánh sát hơn các tình huống sử dụng thực tế.

8.5. HẠN CHẾ

- Khi chuyển sang môi trường đa server, các mô hình yếu gặp khó khăn trong việc quản lý phụ thuộc phức tạp.
- Chi phí đánh giá cao do các mô hình yếu có xu hướng gọi công cụ quá nhiều lần.
- Việc phụ thuộc vào các MCP servers đang hoạt động có thể gặp vấn đề về tính ổn định hoặc thay đổi dữ liệu theo thời gian, dù các server được đánh giá là đạt chuẩn triển khai thực tế.

9. RQ-RAG (Learning to Refine Queries)

9.1. MỤC TIÊU

- Vấn đề với truy vấn RAG:
Các hệ thống RAG hiện tại chủ yếu sử dụng đầu vào ban đầu để truy xuất ngữ cảnh.

Cách tiếp cận này gặp nhiều khó khăn với các truy vấn không rõ ràng, truy vấn phức tạp hoặc truy vấn hội thoại cần nhiều ngữ cảnh, dẫn đến việc truy xuất thông tin không chính xác hoặc gây ảo giác.

- Khi nào cần tinh chỉnh truy vấn:

Việc tinh chỉnh truy vấn là cần thiết trong ba trường hợp chính:

1. Khi truy vấn đơn giản như chào hỏi, không cần truy xuất thông tin.
2. Khi truy vấn phức tạp cần được chia nhỏ thành các câu hỏi con.
3. Khi truy vấn mơ hồ cần được làm rõ ý định để có câu trả lời đầy đủ.

- Ba loại tinh chỉnh truy vấn:

Mục tiêu là trang bị cho mô hình khả năng tự động thực hiện ba hành động sau:

1. Viết lại: Làm rõ truy vấn hoặc điều chỉnh ngữ cảnh cho phù hợp.
2. Phân rã: Chia nhỏ câu hỏi phức tạp thành các bước nhỏ hơn.
3. Làm rõ nghĩa: Xác định đúng ý định người dùng để tạo truy vấn chính xác hơn.

9.2. PHƯƠNG PHÁP

- Phương pháp huấn luyện Llama2 7B:

Tác giả huấn luyện mô hình Llama2 7B theo cách từ đầu đến cuối nhằm giúp mô hình tự động tinh chỉnh truy vấn, thực hiện tìm kiếm và tạo câu trả lời.

- Sử dụng ChatGPT để tạo truy vấn tìm kiếm:

Để xây dựng dữ liệu huấn luyện, nhóm tác giả sử dụng ChatGPT để tạo các truy vấn đã được tinh chỉnh theo nhiều kịch bản khác nhau như viết lại, phân rã và làm rõ. Sau đó mô hình tạo câu trả lời mới dựa trên ngữ cảnh truy xuất được.

- Xây dựng tập dữ liệu với token đặc biệt:

Dữ liệu huấn luyện được xây dựng bằng cách chèn các token đặc biệt nhằm chỉ định loại hành động tinh chỉnh, theo sau là truy vấn đã được tinh chỉnh và các tài liệu truy xuất. Cách làm này biến đổi cặp dữ liệu đầu vào – đầu ra ban đầu thành một chuỗi các bước trung gian có cấu trúc.

- Ba chiến lược lấy mẫu:

Trong giai đoạn suy luận, mô hình sử dụng giải mã dạng cây để tạo nhiều quỹ đạo khác nhau. Ba chiến lược được dùng để chọn kết quả tốt nhất:

1. Perplexity: Chọn quỹ đạo có độ phức tạp thấp nhất.
2. Độ tin cậy: Chọn quỹ đạo có mức độ tin cậy cao nhất cho câu trả lời cuối.
3. Tổng hợp: Chọn câu trả lời có tổng điểm tin cậy cao nhất từ nhiều quỹ đạo.

- Phân tích giới hạn trên:

Tác giả xác định giới hạn trên của hệ thống bằng cách giả định rằng nếu có ít nhất một quỹ đạo dẫn đến câu trả lời đúng thì truy vấn đó được coi là thành công. Kết quả cho thấy tiềm năng rất lớn nếu tồn tại cơ chế lựa chọn quỹ đạo tối ưu.

9.3. KẾT QUẢ

- Câu hỏi một bước:

RQ-RAG vượt trội hơn các phương pháp nền trên nhiều bộ dữ liệu khác nhau. Độ chính xác tăng nhẹ nhưng ổn định so với các mô hình trước đó.

- Câu hỏi nhiều bước:

RQ-RAG thể hiện ưu thế rõ rệt nhờ khả năng phân rã câu hỏi. Hiệu suất tăng mạnh so với mô hình chỉ được tinh chỉnh giám sát, với mức cải thiện trung bình hơn hai mươi phần trăm.

- So sánh với phương pháp trước đó:

Mặc dù sử dụng ít dữ liệu huấn luyện hơn, RQ-RAG vẫn vượt qua phương pháp tiên tiến trước đó trên nhiều tác vụ.

- Mức cải thiện chung:

Hiệu suất trung bình trên các bộ dữ liệu câu hỏi một bước được cải thiện gần hai phần trăm so với phương pháp so sánh.

9.4. ĐIỂM MỚI

1. Chiến lược tinh chỉnh truy vấn:

Đề xuất khung RQ-RAG cho phép mô hình tự động viết lại, phân rã và làm rõ câu hỏi trước khi truy xuất thông tin, thay vì chỉ sử dụng truy vấn ban đầu.

2. Tái tạo câu trả lời dựa trên ngữ cảnh:

Dữ liệu huấn luyện sử dụng câu trả lời mới được tạo dựa trên ngữ cảnh truy

xuất thay vì giữ nguyên câu trả lời gốc. Các thí nghiệm cho thấy cách làm này mang lại hiệu quả cao hơn.

3. Tiềm năng giới hạn trên cao:

Nghiên cứu chỉ ra rằng nếu lựa chọn được quỹ đạo tốt nhất, độ chính xác của hệ thống có thể tăng lên mức rất cao so với kết quả hiện tại.

9.5. HẠN CHẾ

- Khó khăn trong lựa chọn quỹ đạo:

Mặc dù giới hạn trên rất cao, các chiến lược lựa chọn hiện tại vẫn chưa khai thác hết tiềm năng của mô hình, tạo ra khoảng cách lớn giữa kết quả thực tế và lý thuyết.

- Chi phí tính toán:

Việc sử dụng giải mã dạng cây tạo ra nhiều quỹ đạo song song, làm tăng chi phí tính toán so với các phương pháp RAG truyền thống.

- Phụ thuộc vào mô hình giáo viên:

Quá trình tạo dữ liệu phụ thuộc vào ChatGPT, đôi khi gặp các vấn đề như từ chối trả lời hoặc không tuân thủ định dạng, dẫn đến việc phải loại bỏ một phần dữ liệu.

10. CRAG (Corrective RAG)

10.1. MỤC TIÊU

- Vấn đề khi truy xuất thông tin gặp lỗi:

Các hệ thống RAG truyền thống phụ thuộc nhiều vào độ liên quan của tài liệu được truy xuất. Khi bộ truy xuất trả về kết quả không chính xác hoặc không liên quan, mô hình ngôn ngữ lớn sẽ bị nhiễu, dẫn đến hiện tượng ảo giác hoặc cung cấp thông tin sai lệch do không thể tự sửa lỗi dựa trên kiến thức nội tại.

- Cách CRAG giải quyết vấn đề:

CRAG giới thiệu cơ chế tự sửa lỗi nhằm tăng tính mạnh mẽ của quá trình sinh câu trả lời. Hệ thống sử dụng một bộ đánh giá nhẹ để ước lượng chất lượng tài liệu truy xuất và từ đó kích hoạt các hành động phù hợp như lọc kiến thức, tìm kiếm web bổ sung hoặc kết hợp cả hai.

- Mục tiêu chính:

Thiết kế các chiến lược sửa lỗi cho RAG nhằm đảm bảo hệ thống vẫn hoạt động hiệu quả ngay cả khi quá trình truy xuất ban đầu thất bại, đồng thời loại bỏ các ngữ cảnh dư thừa và không hữu ích.

10.2. PHƯƠNG PHÁP

- Bộ đánh giá truy xuất:

Tác giả tinh chỉnh mô hình T5-large với số lượng tham số tương đối nhỏ để làm bộ đánh giá chất lượng truy xuất. Mô hình nhận đầu vào là cặp câu hỏi và tài liệu, sau đó dự đoán điểm liên quan. Dựa trên điểm số này, hệ thống tính toán mức độ tin cậy để quyết định hành động tiếp theo.

- Ba hành động sửa lỗi:

Dựa trên mức độ tin cậy so với các ngưỡng xác định, hệ thống kích hoạt một trong ba hành động sau:

1. Chính xác: Khi độ tin cậy cao, hệ thống sử dụng tài liệu truy xuất và áp dụng quy trình tinh chỉnh kiến thức.
2. Sai: Khi độ tin cậy thấp, hệ thống loại bỏ toàn bộ tài liệu truy xuất và chuyển sang tìm kiếm thông tin trên web.
3. Mơ hồ: Khi độ tin cậy trung bình, hệ thống kết hợp cả kiến thức đã tinh chỉnh từ tài liệu truy xuất và kiến thức thu được từ web.

- Kích hoạt tìm kiếm web:

Tìm kiếm web được kích hoạt khi tài liệu bị đánh giá là sai hoặc mơ hồ. Hệ thống sử dụng mô hình ngôn ngữ để viết lại câu hỏi thành các từ khóa phù hợp và sau đó truy xuất thông tin bổ sung từ nguồn bên ngoài.

- Thuật toán phân rã rồi tái hợp:

Thuật toán này được sử dụng trong quá trình tinh chỉnh kiến thức khi tài liệu được đánh giá là chính xác hoặc mơ hồ.

- Phân rã: Tài liệu được chia thành các đoạn ngắn, mỗi đoạn gồm một đến hai câu.
- Lọc: Bộ đánh giá chấm điểm từng đoạn nhỏ để xác định mức độ liên quan.
- Tái hợp: Các đoạn không liên quan bị loại bỏ, các đoạn liên quan được ghép lại thành một chuỗi kiến thức tinh gọn.

10.3. KẾT QUẢ

- Hiệu suất tổng thể:

Thực nghiệm trên nhiều bộ dữ liệu cho thấy CRAG cải thiện đáng kể độ chính xác và tính đúng đắn của thông tin so với RAG truyền thống.

- So sánh CRAG với RAG:

Trên tất cả các tập dữ liệu đánh giá, CRAG đều đạt kết quả cao hơn RAG chuẩn, đặc biệt rõ rệt trên các tập dữ liệu yêu cầu độ chính xác thực tế cao.

- So sánh Self-CRAG và Self-RAG:

Khi tích hợp CRAG vào khung Self-RAG, hiệu suất tiếp tục được cải thiện rõ ràng trên hầu hết các tập dữ liệu.

- Cải thiện về tính xác thực:

CRAG giúp giảm hiện tượng ảo giác bằng cách loại bỏ thông tin sai lệch từ bước truy xuất và bổ sung kiến thức mới từ web khi cần thiết.

10.4. ĐIỂM MỚI

1. Chiến lược sửa lỗi rõ ràng:

Đây là nghiên cứu đầu tiên đề xuất các chiến lược sửa lỗi cụ thể cho RAG thông qua ba trạng thái chính xác, sai và mơ hồ, thay vì chỉ quyết định có truy xuất hay không.

2. Thiết kế dễ tích hợp:

CRAG có thể được tích hợp trực tiếp vào các hệ thống RAG hiện có mà không cần thay đổi kiến trúc của mô hình sinh câu trả lời.

3. Tối ưu hóa sử dụng tri thức:

Giới thiệu thuật toán phân rã rồi tái hợp nhằm lọc nhiễu trong tài liệu đúng và kết hợp linh hoạt với tìm kiếm web để mở rộng phạm vi kiến thức.

10.5. HẠN CHẾ

- Chi phí tính toán:

CRAG làm tăng nhẹ thời gian thực thi so với RAG chuẩn do bổ sung bước đánh giá và tìm kiếm web. Tuy nhiên, mức tăng này được đánh giá là chấp nhận được so với lợi ích về hiệu suất.

- Phụ thuộc vào bộ đánh giá:

Hiệu quả của hệ thống phụ thuộc nhiều vào độ chính xác của bộ đánh giá truy xuất. Nếu bộ đánh giá đưa ra quyết định sai, hệ thống vẫn có nguy cơ sinh ra thông tin không chính xác, dù hành động mơ hồ đã phần nào giảm thiểu rủi ro.

- Hạn chế với mô hình nền tảng yếu:

Trong các tác vụ đơn giản dạng nhị phân, nếu mô hình sinh câu trả lời có năng lực thấp, hiệu suất tổng thể vẫn có thể không được cải thiện đáng kể dù áp dụng CRAG.

- Yêu cầu tinh chỉnh bổ sung:

CRAG hiện vẫn cần tinh chỉnh một mô hình đánh giá riêng biệt. Hướng nghiên cứu trong tương lai là giảm hoặc loại bỏ sự phụ thuộc này để mô hình ngôn ngữ có thể tự đánh giá tốt hơn.